CloudBurst: Python Implementation Guide

Oct 23, 2026 · @Someone

1. Overview

CloudBurst is built as a self-written Python simulator plus a search algorithm on top of it: a static schedule evaluator (your slide 9 equations), a HEFT scheduler, and a HEFT-seeded PSO that does selective task duplication.

Why Python and not WorkflowSim. WorkflowSim (Java, on CloudSim) has no native boot delay, hourly billing round-up, busy/idle energy or per-task duplication. Adding them means patching its internals. Your model is a static schedule evaluator, so about 500 lines of Python reproduce it exactly, and it runs fast enough to sit inside the PSO loop. Confirm this change with your guide. A defensible line: "WorkflowSim cannot model duplication and busy/idle energy, so we implemented the models of [2] and [3] directly and use the standard Pegasus workflows as input."

Tech stack

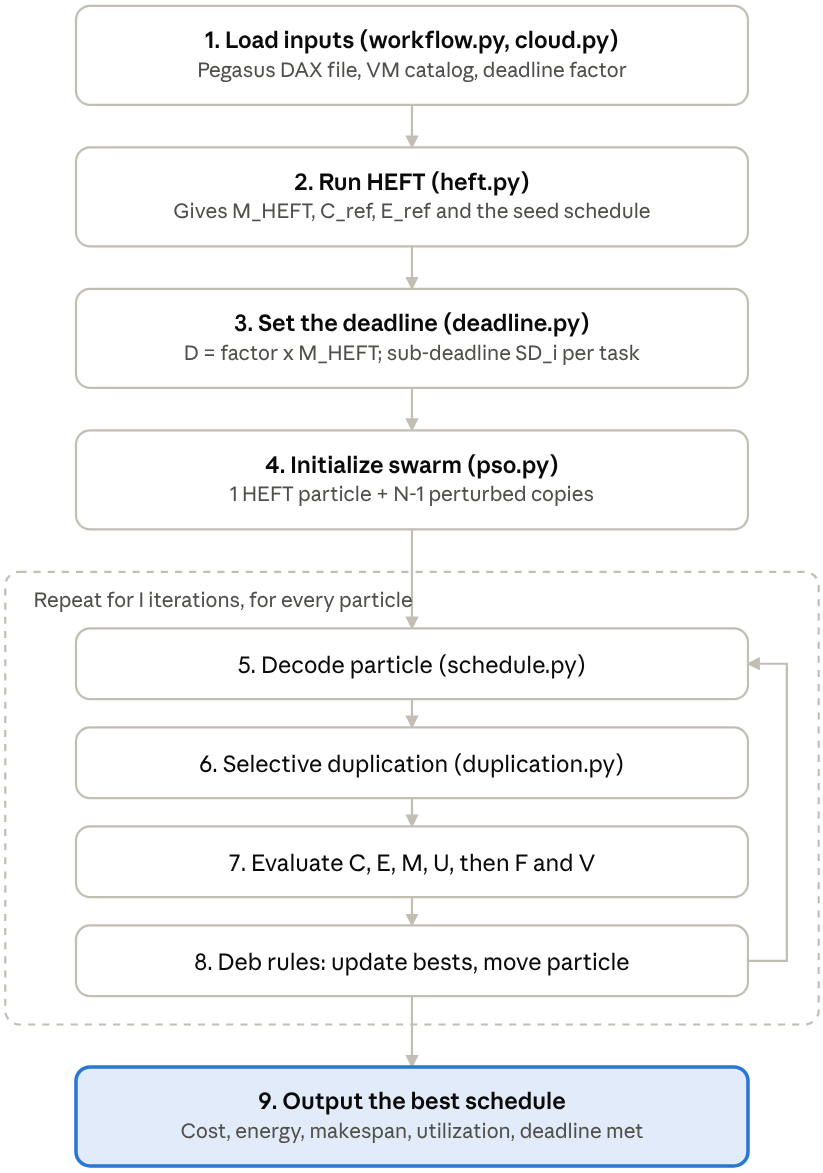
Need	Library
DAG and ranks	`networkx`
Arrays, PSO maths	`numpy`
Speed (only if needed)	`numba`
Results tables	`pandas`
Statistics	`scipy.stats` (Wilcoxon)
Plots	`matplotlib`
Tests	`pytest`

Conventions used everywhere in the code

- Time in seconds, length in MI, speed in MIPS, data in MB, bandwidth in MB/s, price in $ per billing period, power in W, energy in Wh (convert from W·s once, in one place).
- All randomness goes through one `numpy` `Generator` created from a seed passed in. Never call `random` or `np.random` directly, so every run is reproducible.

2. End-to-end flow

One run: HEFT seeds the swarm, then PSO and duplication refine it



one run of CloudBurst · 9 steps, 1 loop

Steps 1 to 4 run once per experiment. Steps 5 to 8 run for every particle in every iteration, so the schedule evaluator (step 5 and 7) is the hot path: make it correct first, then fast. Everything the experiment runner repeats (workflows, sizes, deadlines, algorithms, seeds) wraps around this one run.

3. Project structure

The code splits into four layers. Each layer only imports from the layers above it, so you can test them in order.

```
cloudburst/
  config.yaml            # all parameters: VM types, tau, boot time,
bandwidth, power, PSO settings
  data/workflows/        # Pegasus files: Montage, CyberShake, Epigenomics,
LIGO (50, 100, 200 tasks)
  src/
    workflow.py          # Layer 1: load DAG, MI per task, MB per edge,
upward ranks
    cloud.py             # Layer 1: VM types, VM pool, price, power
    schedule.py          # Layer 2: Schedule object, evaluator -> C, E, M, U
    heft.py              # Layer 3: HEFT
    minmin.py maxmin.py  # Layer 3: Min-Min, Max-Min
    deadline.py          # Layer 3: deadline factors, sub-deadlines
    duplication.py       # Layer 3: selective duplication
    pso.py               # Layer 4: plain PSO and CloudBurst (seeded +
duplication)
  experiments/
    run_all.py           # the grid: workflow x size x deadline x algorithm x
seed
    results/             # one CSV row per run
  analysis/
    tables.py plots.py   # summary tables, convergence curves, Wilcoxon tests
  tests/                 # pytest: one test file per module
```

Layer	Modules	Depends on
1. Data	workflow, cloud	nothing
2. Evaluator	schedule	data
3. Heuristics	heft, minmin, maxmin, deadline, duplication	data, evaluator
4. Search	pso	everything above

The rule that keeps the project simple: every algorithm, whether HEFT, Min-Min or PSO, returns the same `Schedule` object, and only `schedule.py` computes cost, energy and makespan. Then every algorithm is compared by exactly the same code, and a bug in the

accounting is fixed in one place.

4. Modules 1 to 3: data and evaluator

These three modules are the foundation. Spend the most care here, because every result in the project passes through them.

Module 1: `workflow.py`

Job: turn a Pegasus workflow file into numbers the algorithms can use.

```
@dataclass
class Workflow:
    n: int                          # number of tasks, including dummy
entry/exit
    length: np.ndarray              # L_i in MI, shape (n,)
    parents: list[list[int]]        # parents[i] = predecessor task ids
    children: list[list[int]]
    data: dict[tuple[int,int], float]   # d_ij in MB for edge i -> j
    topo: list[int]                 # one valid topological order

def load_dax(path, ref_mips=1000) -> Workflow: ...
def upward_rank(wf, mean_speed, bandwidth) -> np.ndarray: ...
```

- **Parsing.** A Pegasus DAX file lists `job` elements with a `runtime`, `uses` elements (file name, link input or output, size) and `child` / `parent` elements. The newer WorkflowHub files are JSON with similar fields; write the loader for whichever format you download.
- **Length:** `L_i = runtime_i x ref_mips`, using one reference speed for all tasks.
- **Edge data `d_ij`:** the total size of the files that task *i* outputs and task *j* inputs (match by file name).
- **Dummy tasks:** add a zero-length entry and exit task so there is exactly one source and one sink.
- **Upward rank**, computed in reverse topological order, is slide 6's formula:

```
rank_u(t_i) = w_i + \max_{t_j \in succ(t_i)} \left( c_{ij} + rank_u(t_j)
\right), \quad w_i = L_i / \overline{s}
```

- **Check:** print the task count, edge count, critical path length and total data for each file, and compare with the published statistics of that workflow.

Module 2: `cloud.py`

Job: describe the VM catalog and the pool of VM instances the algorithms choose from.

```
@dataclass(frozen=True)
class VMType:
    name: str
    speed: float      # MIPS
    price: float      # $ per billing period
    p_busy: float     # W
    p_idle: float     # W

TAU = 3600.0          # billing period (s)
T_BOOT = 97.0         # boot time (s), from [2]
BANDWIDTH = 100.0     # MB/s between any two VMs

def build_pool(types, per_type) -> list[int]:
    """Pool = per_type instances of every type; returns the type index of
each instance."""
```

Starting values (these are assumptions: replace them with figures you can cite, such as EC2 instance specs, and say so in the report):

Type	Speed (MIPS)	Price ($/period)	P_busy (W)	P_idle (W)
small	1000	0.10	100	65
medium	2000	0.19	170	110
large	4000	0.36	300	190
xlarge	8000	0.70	520	330

Keep price growing slower than speed, and keep `p_idle` at about 60 to 70% of `p_busy`. Both are common modelling assumptions. Put everything in `config.yaml` so sensitivity tests only change the config.

Pool size: `per_type` of about 2 to 3 times the widest level of the DAG is enough. A huge pool slows PSO, and a tiny pool limits it. Rodriguez and Buyya found the result sensitive to this, so you will report it as a parameter.

Module 3: `schedule.py` (the core)

Job: given an assignment of tasks to VM instances, simulate it and report cost, energy, makespan and utilization. This implements slide 9.

```
@dataclass
class Placement:
    task: int
    vm: int            # index into the pool
    start: float
    finish: float
    is_dup: bool = False

@dataclass
class Schedule:
    placements: list[Placement]   # primaries and duplicates
    cost: float; energy: float; makespan: float; util: float

def evaluate(wf, pool, assign, order, dup_fn=None) -> Schedule: ...
```

Algorithm of `evaluate`:

1. Process tasks in `order` (decreasing upward rank, which is always a valid topological order).
2. For task *i* on VM *k*, `data_ready = max over parents j of ( AFT(j) + TT(j,i) )`, where `TT = d_ji / BANDWIDTH` and is 0 if parent *j* ran on the same VM (or a duplicate of *j* is on this VM).
3. If this is the VM's first task, set `lease_start = max(0, data_ready - T_BOOT)` (boot overlaps the wait) and the VM is ready at `lease_start + T_BOOT`.
4. `EST = max(vm_free_time, data_ready, vm_ready)` and `AFT = EST + L_i / s_k`.
5. Optionally call `dup_fn` here (module 7), which may add a duplicate and lower `AFT`.
6. After all tasks: for each used VM, `lease_end = last finish`. Cost, energy and utilization follow:

```
C = \sum_k \lceil (lease\_end_k - lease\_start_k) / \tau \rceil \cdot p_k
\qquad
E = \sum_k \left( P^{busy}_k T^{busy}_k + P^{idle}_k (lease_k - T^{busy}_k)
\right) \qquad
U = \frac{\sum_k T^{busy}_k}{\sum_k lease_k}
```

where `T_busy` counts every execution, **duplicates included**, and `lease` includes boot time. `M = AFT(exit)`.

Decisions to write down in the report (the paper's choices differ, and an examiner will ask): boot time is paid and billed; an unused VM costs nothing; the lease ends at the last task finish; transfers are assumed not to delay lease end; the network is a single flat bandwidth.

Unit tests (write these before the code):

- 3-task chain on one VM: makespan = `T_BOOT + sum(L)/s`, cost = 1 period if under 3600 s.
- Fork-join with parents on different VMs: the child waits for the slower transfer.
- Cost rounding: a lease of 3601 s costs 2 periods.
- Energy: a VM idle between two tasks pays `p_idle` for the gap.
- A duplicate adds to busy time, energy and (if it extends the lease) cost. It is never free.

5. Modules 4 and 5: baselines and deadlines

A design point before you write HEFT

HEFT, Min-Min, Max-Min and your duplication step all need to ask "if I put task *i* on VM *k*, when does it start and finish?" without committing. So build `schedule.py` around a **`ScheduleBuilder`** class with two methods, and let `evaluate` just call them in a loop:

```
class ScheduleBuilder:
    def __init__(self, wf, pool): ...
    def try_place(self, task, vm) -> tuple[float, float]:   # (est, aft),
changes nothing
    def commit(self, task, vm, est, aft): ...               # records the
placement
    def finish(self) -> Schedule:                           # computes C, E,
M, U
```

This keeps one copy of the timing logic, which is where the bugs hide.

Module 4: `heft.py`, `minmin.py`, `maxmin.py`

HEFT (slide 6) is the most important baseline: it is also the seed of CloudBurst.

1. Compute `rank_u` for every task and sort by decreasing rank.

2. For each task in that order, call `try_place` on every VM in the pool and keep the VM with the smallest `aft`. Ties go to the cheaper type.
3. `commit` it. Return `builder.finish()` plus the assignment array `assign[task] = vm`.

Plain HEFT ignores cost, so with a big pool it spreads tasks over many fast VMs and pays for many partly used billing periods. That is the exact weakness CloudBurst exploits, so expect HEFT to have the lowest makespan and a high cost.

Min-Min / Max-Min:

1. Keep the set of ready tasks (all parents done).
2. For each ready task find its best VM and the resulting earliest finish time.
3. Min-Min commits the ready task whose best finish time is smallest. Max-Min commits the one whose best finish time is largest.
4. Repeat until all tasks are placed.

Optional, if time allows (slide 11 says "where feasible"): HEFT-TD for [1] is HEFT plus a step that tries duplicating the critical parent onto each candidate VM and keeps it only if the finish time improves. A simplified CEAS for [3] is a deadline split, cheapest VM that meets each sub-deadline, then merging. If you implement a simplified version, call it that in the report.

Module 5: `deadline.py`

Deadline generation. Run HEFT first and take its makespan `M_HEFT`. Then test several deadlines from tight to loose:

```
D = f \cdot M_{HEFT}, \qquad f \in \{1.1,\ 1.5,\ 2,\ 3\}
```

Because HEFT is close to the fastest possible makespan, a `f` near 1 is a tight deadline and a large `f` is loose. Run a first test and calibrate the factors so that, at the tightest setting, some algorithms fail and at the loosest, all pass. Otherwise deadline success rate tells you nothing. An alternative from [2] is to place `D` between the fastest and the slowest possible makespan, which you can mention as future work.

Sub-deadlines (slide 9):

```
SD_i = AFT_{HEFT}(i) \cdot D / M_{HEFT}
```

This scales every task's HEFT finish time up to the deadline, so a task with slack can move to a cheaper, slower VM. Your pseudocode computes `SD_i` but does not use it in the fitness. Decide how it is used, and write it in the report. Two options:

- **Option A (recommended first):** use `SD_i` only as a guide inside duplication and for reporting how many tasks miss their sub-deadline.
- **Option B (ablation later):** add a small penalty `sum_i max(0, AFT_i - SD_i) / D` to `V`, so infeasible particles that are "nearly on time" rank better.

Start with A. If time allows, run B as an extra ablation and report whether it helps.

6. Modules 6 and 7: PSO and selective duplication

This is the part that is yours. Build the plain PSO first, then seeding, then duplication, and test each step against the previous one.

Module 6: `pso.py`

Encoding (slide 7). A particle is a vector `x` of length `n` (one coordinate per task) with each value in `[0, V)`, where `V` is the pool size. Task *d* runs on VM `floor(x[d])`. Task order is *not* searched: it is fixed to decreasing upward rank, so every particle decodes to a valid schedule.

Parameters (starting values, tune on small workflows only):

Parameter	Value
Swarm size N	30 to 50
Iterations I	100 to 200
Inertia w	0.9 falling linearly to 0.4
c1, c2	2.0, 2.0
Velocity clamp	+/- V / 4
Perturbation rate for seed copies	10 to 20% of tasks
Weights (w1, w2, w3)	(0.4, 0.3, 0.3) first, then a sensitivity run

Initialization:

- **Plain PSO (baseline):** every particle has random `x` in `[0, V)`.
- **CloudBurst:** particle 0 is the HEFT assignment, `x[d] = assign[d] + 0.5` (the 0.5 keeps `floor` stable). Particles 1 to N-1 copy it, then re-draw 10 to 20% of the coordinates at random.

Fitness and Deb's rules. Each particle gets `F` (normalised objective), `V` (deadline violation) and a feasibility flag:

```
F = w_1 \frac{C}{C_{ref}} + w_2 \frac{E}{E_{ref}} + w_3 \frac{M}{M_{ref}},
\qquad V = \max(0,\ M - D)
```

```
def better(a, b):
    """Deb's rules: is solution a better than b?"""
    if a.V == 0 and b.V > 0: return True       # feasible beats infeasible
    if a.V > 0 and b.V == 0: return False
    if a.V == 0 and b.V == 0: return a.F < b.F  # both feasible: lower F
    return a.V < b.V                            # both infeasible: lower
violation
```

Use `better` for **every** comparison: updating `pbest`, updating `gbest`, and choosing the final answer.

Main loop:

```
def cloudburst(wf, pool, D, refs, rng, seeded=True, dup=True, N=40, I=150):
    X, Vel = init_swarm(wf, pool, rng, seeded)        # HEFT seed + perturbed
copies
    pbest = [evaluate_particle(x) for x in X]
    gbest = best_of(pbest)
    history = []
    for it in range(I):
        w = w_max - (w_max - w_min) * it / I
        for p in range(N):
            r1, r2 = rng.random(n), rng.random(n)
            Vel[p] = w*Vel[p] + c1*r1*(pbest[p].x - X[p]) + c2*r2*(gbest.x -
X[p])
            Vel[p] = np.clip(Vel[p], -vmax, vmax)
            X[p] = np.clip(X[p] + Vel[p], 0, V - 1e-9)
            cand = evaluate_particle(X[p], dup=dup)    # decode + duplication
+ F, V
            if better(cand, pbest[p]): pbest[p] = cand
            if better(cand, gbest):    gbest = cand
        history.append(gbest.F)                         # for convergence
curves
    return gbest, history
```

Always save `history` (best `F` per iteration): the convergence curves prove the HEFT-seeding claim on slide 10. Because the HEFT particle is in the swarm from iteration 0, `gbest` can never be worse than HEFT. Check this in a test.

Module 7: `duplication.py`

Idea (slide 8): while placing task *i* on VM *k*, a copy of its slowest-arriving parent on that VM may remove a data transfer. Keep the copy only if it pays off.

For each task *i* in order, after choosing VM *k*:

1. Find the critical parent *p*: the parent maximising `AFT(p) + TT(p, i)` whose primary copy is on a different VM.
2. Using `try_place`, compute where a copy of *p* would run on VM *k*: it starts at `max(vm_free(k), data for p's own parents to reach k)` and takes `L_p / s_k`.
3. Recompute the start and finish of *i* with the copy present (its data from *p* is now local, so the transfer is 0).

4. **Accept** only if all three hold: `AFT'(i) < AFT(i)`; the extra cost is within the threshold, preferring time that is already inside a paid billing period; the extra energy is within the threshold.
5. If accepted, `commit` the duplicate with `is_dup = True`. Otherwise do nothing.

Thresholds: start with `extra billing periods = 0` (the copy must fit inside time you already pay for) and `extra energy <= 50%` of the task's own energy. Make both parameters, and test a few values.

Where this goes wrong:

- A duplicate that is not counted in busy time, energy or lease end looks like free speed-up. Test this explicitly.
- Do not duplicate onto a VM before its boot has finished.
- A copy only helps tasks that run later on that VM, so a parent copied for task *i* may also serve other children on the same VM. Allow the builder to reuse an existing copy.
- The first version can be append-only (the copy runs at the end of VM *k*'s current schedule). Insertion into earlier idle gaps is an optional improvement.

7. Experiments and analysis

`experiments/run_all.py`

One function runs one (workflow, size, deadline factor, algorithm, seed) combination and appends one row to a CSV. The grid is a set of nested loops around it, and results are never kept only in memory, so a crash loses nothing.

```
for wf_name in ["montage", "cybershake", "epigenomics", "ligo"]:
    for size in [50, 100, 200]:
        wf = load_dax(f"data/workflows/{wf_name}_{size}.dax")
        heft_sched = heft(wf, pool)                    # gives M_HEFT,
C_ref, E_ref
        for f in [1.1, 1.5, 2, 3]:
            D = f * heft_sched.makespan
            for algo in ALGOS:
                for seed in (range(30) if algo.randomized else [0]):
                    t0 = time.time()
                    s = algo.run(wf, pool, D, refs, seed)
                    append_row(wf_name, size, f, algo.name, seed, s,
time.time() - t0)
```

Columns per row: workflow, size, deadline factor, D, algorithm, seed, makespan, cost, energy, utilization, deadline met (yes/no), scheduling time, number of duplicates.

Algorithms to compare

Name	What it is	Which claim it tests
HEFT, Min-Min, Max-Min	Classic heuristics	Baselines
PSO-plain	Random start, no duplication	Rodriguez-style baseline [2]
PSO-seeded	HEFT seed, no duplication	Advantage 1: seeding reduces variability and slowness
PSO-dup	Random start, with duplication	Advantage 2: duplication under cost and energy limits
CloudBurst	HEFT seed + duplication	The full method
HEFT-TD, CEAS-lite (optional)	Simplified versions of [1] and [3]	Comparison with related work

The PSO rows (PSO-plain, PSO-seeded, PSO-dup and CloudBurst) form your **ablation study**. They are the strongest evidence that each part of the method does something.

Metrics and how to report them

- **Makespan, cost, energy, utilization**, as mean and standard deviation over the 30 seeds.
- **Deadline success rate:** the share of runs with `M <= D`.
- **Normalised score F** (relative to HEFT, so 1.0 means "same as HEFT").
- **Scheduling time** in seconds. Slide 10 says CloudBurst will be slower than the heuristics, so report that honestly.
- **Convergence:** mean best-`F` per iteration for PSO-plain, PSO-seeded and CloudBurst on a few workflows.
- **Statistics:** for each claim "A is better than B", run a Wilcoxon signed-rank test (`scipy.stats.wilcoxon`) over paired runs, and report the p-value.

Output of `analysis/`

1. A results table per workflow: rows = algorithm, columns = cost, energy, makespan, utilization, success rate.
2. Bar charts of cost and energy relative to HEFT for each deadline factor.
3. Convergence curves (seeded versus random start).
4. A box plot of cost across the 30 seeds, to show run-to-run variability.
5. A sensitivity plot for the weights and for the duplication thresholds.

Rules for trustworthy results: fix every seed and log it; tune PSO parameters on small workflows, then freeze them before the main runs; use the same pool and the same cloud config for every algorithm; if CloudBurst loses on some workflow, report it and say why.

8. Testing, build order and viva preparation

Build order

Build in this order and do not start a phase until the "done when" check of the previous one passes. Expect roughly 2 to 3 weeks of part-time work with an AI coding assistant, and longer without one.

Phase	Build	Done when
1	`workflow.py`, `cloud.py`, `ScheduleBuilder` and `evaluate`	Hand-calculated chain and fork-join tests pass
2	`heft.py`, `minmin.py`, `maxmin.py`, `deadline.py`	HEFT prints a sensible schedule for one Montage file; cost, energy and makespan are plausible
3	Plain PSO, then HEFT seeding	Seeded `gbest` is never worse than HEFT; convergence curve is smooth
4	`duplication.py`	Duplicates raise energy and busy time; makespan drops on a DAG with heavy transfers
5	`run_all.py` on small workflows, then the full grid	CSV has every row, no crashes, seeds logged

Phase	Build	Done when
6	`analysis/`, plots, report and slides	Every claim on slide 10 has a table, a plot and a p-value

Tests to write (pytest)

- **Evaluator:** the five tests listed under Module 3.
- **HEFT:** its order is a valid topological order; on a 1-VM pool the makespan equals the sum of task times plus boot.
- **PSO:** with the HEFT seed, the final `F <= 1.0` and the result is feasible whenever `D >= M_HEFT`; two runs with the same seed give identical output.
- **Duplication:** a duplicate is counted in cost, energy and busy time; with thresholds set to 0 the result equals the no-duplication schedule.
- **Invariants on any schedule:** every task is placed at least once; no task starts before all its parents finish plus transfer; no two tasks overlap on a VM.

Common pitfalls

- **Speed:** PSO decodes a full schedule per particle per iteration (for example 40 particles x 150 iterations x 200 tasks is about 1.2 million placements). Use `numpy` arrays, avoid Python objects in the inner loop, and use `numba` if needed. Start on 50-task workflows.
- **Unit mix-ups:** keep seconds, MB, MIPS and W in the conventions of section 1, and convert W·s to Wh in one place.
- **Billing rounding** applied per VM, not per task.
- **Using the test workflows to tune parameters.** Tune on a few small ones, freeze, then run the grid.
- **Reporting only the mean.** Always include the standard deviation and the success rate.

Questions you should be able to answer

1. Why is a custom Python simulator valid instead of WorkflowSim, and what did you take from the papers to make it comparable?
2. How is a particle decoded into a schedule, and why is the task order not part of the particle?
3. Why do Deb's rules compare infeasible solutions by violation and not by cost?

4. When does duplication reduce the makespan, and when would it raise the cost? How do you stop that?
5. Why is the HEFT seed guaranteed to give a result no worse than HEFT?
6. What assumptions did you make about boot time, billing, bandwidth and idle power, and how would the results change if they were different?
7. Which results did not favour CloudBurst, and why?